IA EMPRESARIAL · CLAUDE CODE

El playbook del SDLC nativo de IA

Cómo transformar tu ciclo de vida de desarrollo de software con IA, etapa por etapa.

Autor: Louis Claxton **Fecha:** 21 de agosto de 2026 **Lectura:** 46 min

Producto: Claude Enterprise, Claude Code, Claude Tag

> Traducción completa al español del artículo original de Anthropic, «The AI-Native SDLC playbook», incluyendo todos sus diagramas originales. Los bloques de código y configuración se mantienen en su forma técnica original (inglés) porque reflejan sintaxis real de herramientas; el resto del contenido —incluidos los documentos de ejemplo tipo `intent.md`— está traducido.

El código ya no es el cuello de botella

Las organizaciones han empezado a usar IA para escribir código a una velocidad impensable hace apenas un año, pero los procesos que rodean a ese código no han cambiado al mismo ritmo.

Muchos equipos de ingeniería siguen manteniendo las mismas puertas de aprobación, revisiones, traspasos y políticas de siempre, frenando las ganancias de productividad que traen las soluciones de codificación agéntica como Claude Code.

El ciclo de vida de desarrollo de software (SDLC, por sus siglas en inglés) es el proceso que lleva al software desde la idea hasta producción. La mayoría de las organizaciones ejecutan alguna variante de las mismas seis etapas: planificación, diseño, construcción, pruebas, despliegue y mantenimiento. Tradicionalmente, cada etapa es una fase independiente a cargo de un rol distinto. Los product managers redactan los requisitos, los arquitectos técnicos los convierten en diseños, los ingenieros construyen esos diseños, los equipos de QA de las empresas reguladas los verifican, los equipos de release los publican, y operaciones monitoriza lo que está en producción. El trabajo se mueve entre fases a través de documentos, tickets y firmas de aprobación.

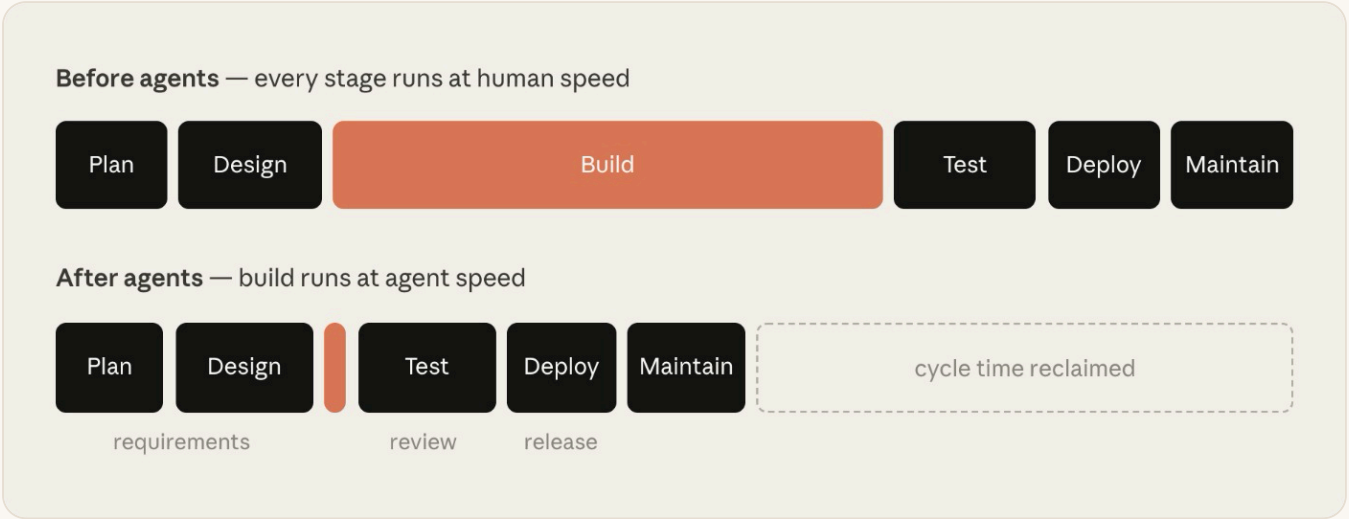
El SDLC tradicional es intensivo en procesos para garantizar responsabilidad y control en cada paso. Sin embargo, fue diseñado para maximizar la eficiencia en una época en la que la etapa más costosa y lenta era escribir e implementar código, algo que ya no es cierto. Los PRD, los rituales de estimación y las revisiones de seguridad de producto existían para forzar la alineación durante lo que podían ser semanas, meses o trimestres de desarrollo.

El SDLC tradicional también incluye controles que asumen que cada paso lo ejecuta una persona. Las organizaciones que generan más valor han reconstruido su proceso alrededor de lo que la IA agéntica puede hacer ahora, asegurándose de que el ser humano siga presente en el bucle. En esta guía repasamos varias de las mejores prácticas de nuestro equipo de Applied AI para integrar Claude internamente en cada etapa del SDLC, acelerar el desarrollo y hacer que los procesos avancen más rápido, inspiradas en el trabajo con nuestros clientes.

Cuando el código deja de ser el cuello de botella y la fase de construcción avanza más rápido de lo que el SDLC tradicional permite, se cumplen tres cosas:

- El cuello de botella se desplaza a los pasos que están antes y después de la construcción: sobre todo planificación, revisión/pruebas y despliegue, que siguen funcionando a velocidad humana.
- Los controles dejan de encajar con la realidad y se vuelven inmanejables. Revisar cada línea a mano tenía sentido cuando una persona la había escrito, pero no aguanta el ritmo cuando los agentes escriben la mayor parte del diff.
- El coste de la gobernanza aumenta porque las excepciones siguen pasando por reuniones y comités que se reúnen semanal o mensualmente.

La construcción ya no es la restricción: lo son los pasos a velocidad humana que la rodean. Las etapas que dependen de personas mantienen su duración mientras la construcción se reduce a horas.



Antes de los agentes: todas las etapas corren a velocidad humana. **Después de los agentes:** la construcción corre a velocidad de agente, y el tiempo de ciclo recuperado queda disponible para el resto del proceso — planificación (requisitos), revisión y despliegue (release) siguen dependiendo de personas.

Tomemos un cuello de botella de seguridad como ejemplo. Los equipos de seguridad están dimensionados para un rendimiento humano, así que cuando los agentes multiplican la producción de código, o la cola de revisión se satura o el código se publica sin revisión suficiente. Una organización

regulada no puede aceptar ninguno de los dos resultados, así que sus controles de seguridad y de políticas tienen que seguir el ritmo de los agentes.

Para aprovechar mejor las ganancias de productividad y asegurar la IA agéntica, el SDLC tradicional necesita el mismo nivel de transformación que ya ha vivido la fase de implementación.

ÍNDICE

1. El código ya no es el cuello de botella
2. Plays
3. Etapa 1 — Plan
4. Etapa 2 — Design
5. Etapa 3 — Build
6. Etapa 4 — Test
7. Etapa 5 — Deploy
8. Etapa 6 — Maintain
9. Reflexiones finales

¿Qué es un SDLC nativo de IA?

El SDLC nativo de IA es un proceso reimaginado que combina los objetivos de control de siempre con una nueva forma de aplicarlos. En lugar de un flujo lineal, el proceso se convierte en un bucle, y la IA queda integrada en cada punto. El SDLC nativo de IA promueve el traspaso automatizado y el disparo de las siguientes fases, atacando así la naturaleza manual y torpe del traspaso entre fases del SDLC tradicional.

A este cambio también se le llama SDLC agéntico, SDLC de IA, o simplemente desarrollo de software agéntico: las etiquetas cambian, pero describen lo mismo.

Los cambios en las seis etapas de un SDLC nativo de IA

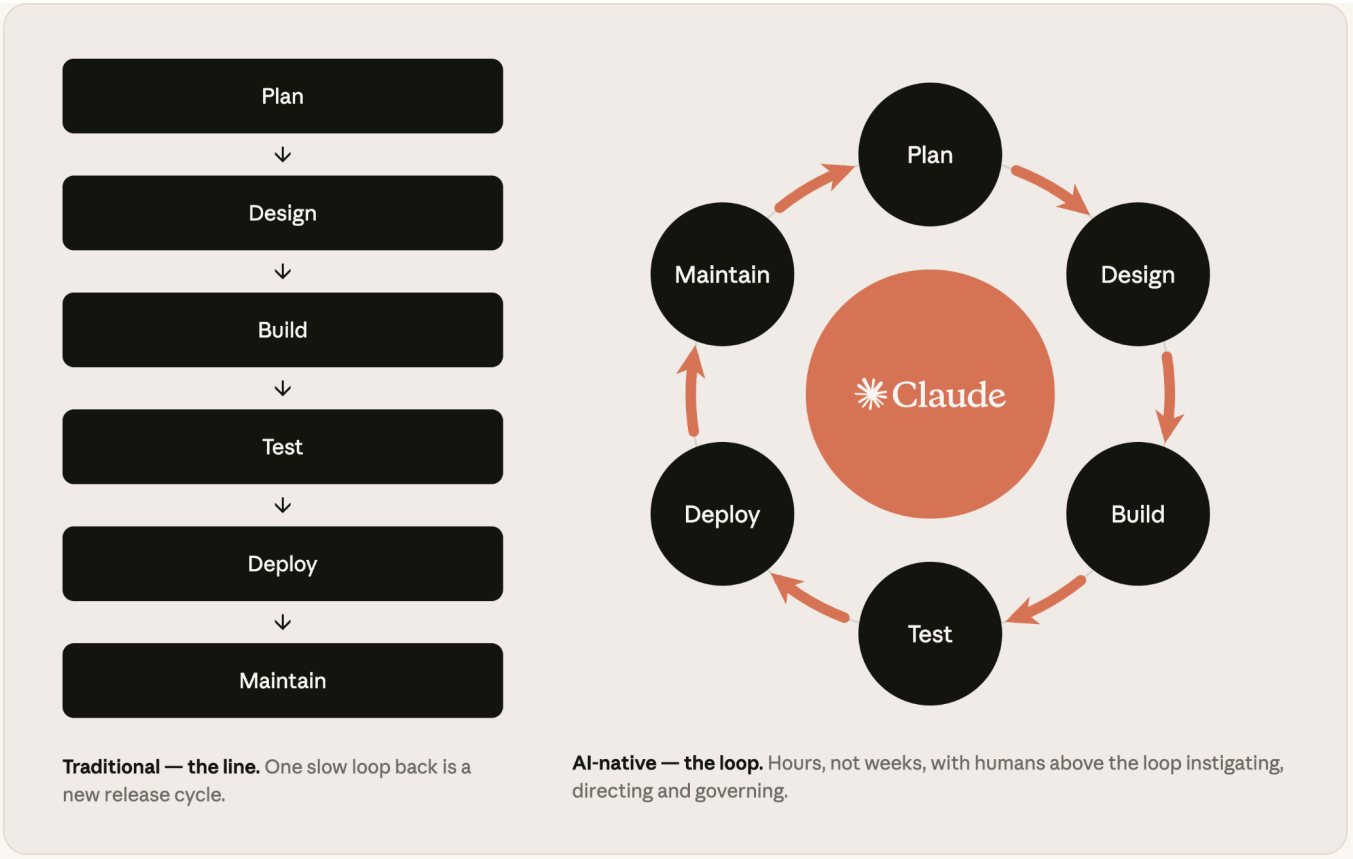
La tabla siguiente muestra los extremos del espectro entre el SDLC tradicional y el SDLC nativo de IA con soporte de Claude. La mayoría de las organizaciones se sitúan en algún punto entre ambas columnas.

ETAPA	SDLC TRADICIONAL	SDLC NATIVO DE IA
Plan	Los requisitos se reúnen por comité, se destilan en talleres y firmas de aprobación, y se redactan a mano.	Claude sintetiza los puntos de dolor directamente desde las fuentes y los recoge en un `intent.md` legible por humanos y accionable por máquinas.
Design	La especificación la escriben analistas y la interpretan diseñadores.	Requisitos y diseño se comprimen en una sola sesión de trabajo con un agente, guiada por estándares codificados como skills y versionada en git.
Build	Pruebas y código se escriben a mano, y la documentación se redacta después del desarrollo principal.	Pruebas y código los genera la IA, y el conocimiento institucional se mantiene como archivos `CLAUDE.md` versionados y legibles por máquina, junto con skills.
Test	Puertas de QA en los límites de cada etapa.	Evaluaciones continuas entretejidas con la implementación.
Deploy	Los humanos revisan cada línea de código y la gobernanza ocurre en ciclos de revisión, a menudo de forma inconsistente.	Capas de revisión agéntica, con revisión humana reservada para código regulado o crítico. La gobernanza se aplica mientras la IA actúa, con hooks como puertas de aprobación.
Maintain	Los humanos vigilan producción en busca de errores.	Los agentes monitorizan los despliegues activos. Cualquier banda de control incumplida se diagnostica y se reincorpora al bucle como un nuevo `intent.md`.

El hilo conductor de la columna derecha es el artefacto confirmado (*commit*). Cada etapa termina escribiendo uno en el control de versiones — `intent.md`, `spec.md`, `plan.md`, el diff con sus pruebas, el PR con sus hallazgos de revisión, y el registro del incidente— y la siguiente etapa empieza leyéndolo. En las primeras etapas, los archivos `.md` son el artefacto predominante porque tanto un product owner como un agente pueden leerlos y actuar sobre ellos. A partir de Build, el artefacto es el código y sus registros. La cadena de commits es también el rastro de auditoría: quién pidió qué, qué produjo el agente, y quién lo aprobó.

Los humanos siguen siendo responsables de toda decisión que requiera criterio. En el mundo del SDLC agéntico, la atención humana se desplaza junto con los artefactos que hay que revisar.

Cada etapa confirma un artefacto que la siguiente etapa puede leer. Juntos, el intent, el spec, el plan, el diff y los hallazgos de revisión forman el rastro de auditoría.



Tradicional — la línea. Un único bucle de retorno, lento, inicia un nuevo ciclo de release. **Nativo de IA — el bucle.** Horas, no semanas, con las personas por encima del bucle iniciando, dirigiendo y gobernando el proceso.

Plays

Los plays son el núcleo de este playbook y se agrupan en seis etapas no lineales (Plan, Design, Build, Test, Deploy, Maintain) que en conjunto cubren todo el ciclo de vida.

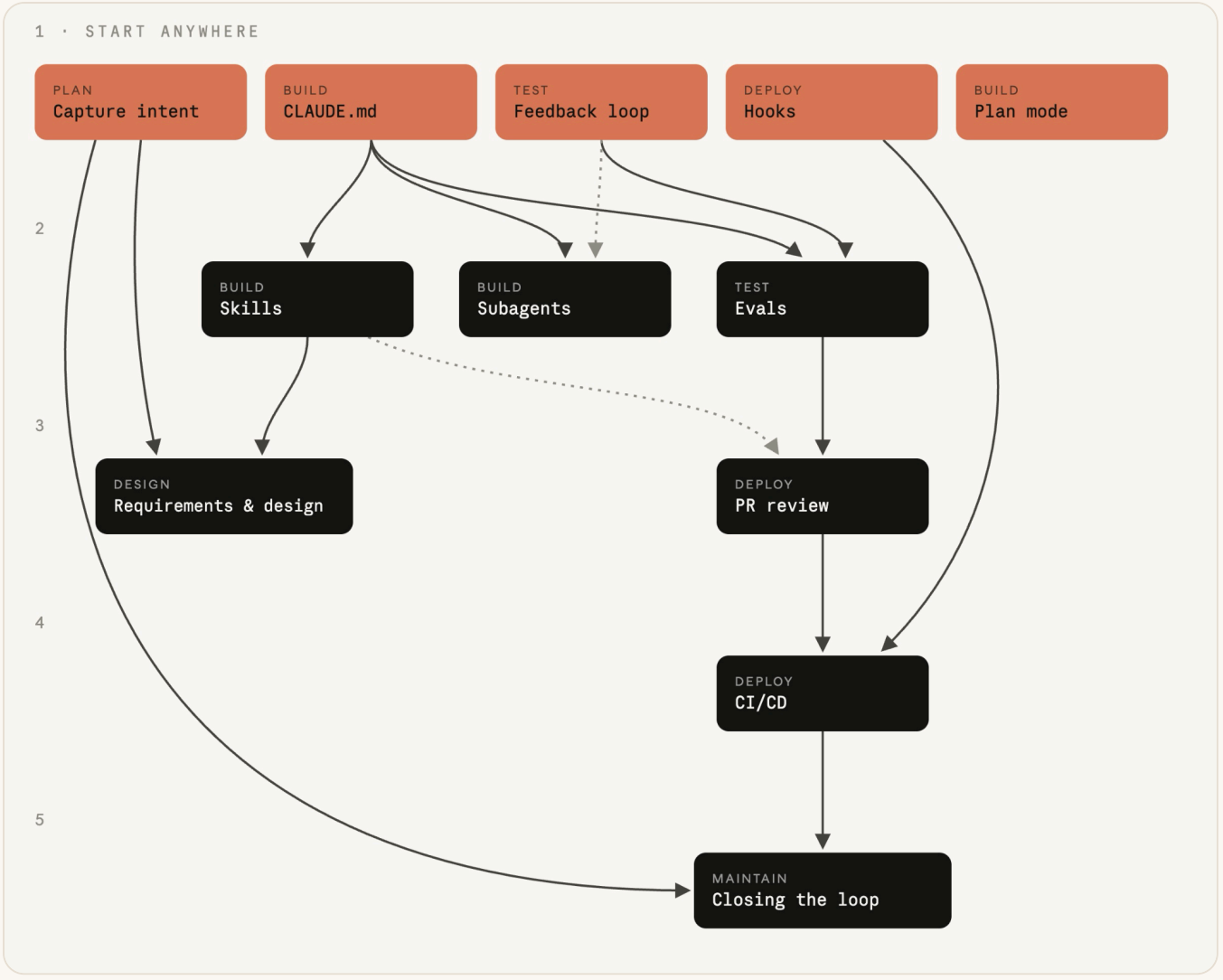
Cada play cubre:

- Qué cambia;
- Cómo empezar;
- Pasos concretos de implementación;
- Consideraciones de gobernanza; y
- Cómo medir si funcionó.

Los pasos son modulares, y cada organización puede priorizar la transformación de distintas etapas en distintos momentos según sus necesidades particulares. Cada play nombra sus dependencias bajo «Requisitos previos», que el grafo de dependencias ilustra más abajo.

Una etapa termina confirmando un artefacto, y ese commit da inicio a la siguiente etapa. Un `intent.md` aceptado dispara el pase de requisitos y diseño, un `spec.md` aprobado dispara el modo plan, un PR fusionado dispara el pipeline, y una banda de control incumplida en producción escribe el siguiente `intent.md`, y así continúa el bucle.

Al principio, cada paso se dispara a mano, con el estado final siendo un bucle en el que cada artefacto aceptado dispara la siguiente puerta. La atención humana se concentra en esas puertas, revisando lo que el agente señaló en vez de arrancar cada etapa desde cero.



Los plays listados por etapa; las flechas indican el orden recomendado de adopción — no es lo mismo que el orden de las etapas. Se puede empezar por cualquier play de color naranja (nivel 1), ya que nada apunta hacia ellos y no necesitan nada antes. Para cualquier otro play, las flechas que entran en él son los plays que conviene adoptar primero.

01 Plan

Las ideas dejan de esperar a que alguien las redacte. La intención se captura una sola vez, en las propias palabras de quien la origina, como un artefacto versionado sobre el que la siguiente etapa puede actuar.

Capturar como intent.md

El `intent.md`, que da el pistoletazo de salida al proceso de desarrollo, puede entrar por distintas vías. Una persona tiene una idea, se abre un ticket, o surge un incidente a través de una alerta (ver Etapa 6: Maintenance).

Cuando alguien tiene una idea, hace una lluvia de ideas con Claude y produce un proto-spec en markdown. En el SDLC tradicional, esa misma persona tendría que convencer a alguien del equipo de producto para que redactara la idea junto a ella o en su nombre.

El proto-spec que genera Claude es legible por humanos, está versionado, y es inmediatamente consumible por la siguiente etapa. El proto-spec se guarda como `intent.md`.

Sin importar si la intención se origina por un disparador de evento o por un agente, se aplican los mismos pasos: el product owner revisa y corrige el `intent.md` escrito por el agente antes de confirmarlo.

Tradicional

Una idea pasa por entradas de backlog, historias de usuario, story points y reuniones de refinamiento antes de que nadie pueda actuar sobre ella. La propiedad se transfiere en cada traspaso, así que lo que llega a ingeniería está a varios pasos de distancia de lo que quería decir quien la originó.

Nativo de IA

Quien origina la idea hace una lluvia de ideas con Claude y escribe el resultado como `intent.md`, un proto-spec en sus propios términos. El artefacto contiene qué se quiere, por qué, y bajo qué restricciones. Los procesos repetidos se codifican mediante skills.

CÓMO EMPEZAR

REQUISITOS PREVIOS

Ninguno.

INFRAESTRUCTURA

Acceso a Claude para personas que no son ingenieras (claude.ai o Cowork); una plantilla de `intent.md` acordada; un lugar compartido y versionado para la intención que el product owner supervise. Para un solo producto, el lugar más sencillo es una carpeta `intent/` dentro del repositorio del producto: esto mantiene la cadena de artefactos junto al código que deriva de ella. Un repositorio de intención dedicado solo compensa el esfuerzo cuando la intención abarca muchos repositorios, y en un monorepo es simplemente un directorio. El recuadro de la Etapa 3: Build explica cómo se relaciona este lugar con una herramienta como Jira que ya guarda el registro.

Configurar esto es una tarea de una sola vez para el equipo de plataforma o ingeniería. Una persona técnica del equipo tiene que levantar el repositorio de intención y decidir quién puede escribir en él, ya que muchas personas contribuyentes vendrán de toda la organización.

Una vez existe el repositorio, quienes contribuyen sin experiencia en git no necesitan usarlo directamente. En su lugar, un conector al sistema de control de versiones (por ejemplo, GitHub) permite que Claude confirme archivos markdown en su nombre desde claude.ai o Cowork.

CÓMO EJECUTARLO

1. Quien origina la idea describe el problema a Claude con sus propias palabras. Puede describir qué no puede hacer hoy, a quién afecta la idea, cómo sería mejor, o qué queda fuera de alcance. No hace falta lenguaje formal.
2. Se hace una lluvia de ideas hasta que la idea sea concreta. Claude hace las preguntas que haría una persona analista: alcance, usuarios, restricciones y qué aspecto tiene el éxito.
3. Se le pide a Claude que escriba el resultado como `intent.md` usando la plantilla de la organización, que puede codificarse como una skill configurada por una persona técnica y aprobada por un lead. Puede cubrir el problema, el resultado propuesto, los usuarios y sistemas afectados, las restricciones y las preguntas abiertas.
4. Quien originó la idea corrige cualquier malentendido de Claude.
5. Se confirma el `intent.md` en el lugar compartido. La autoría y la marca de tiempo se unen al registro, y el product owner recoge la idea desde ahí.

Ejemplo de intent.md

```
# Intent: autoservicio de estado de siniestros

Autor: J. Ortiz (operaciones de siniestros). Estado: borrador.

## Problema
Los clientes llaman al centro de contacto para preguntar por el estado
de su siniestro. El personal de atención dedica aproximadamente un
tercio del tiempo de llamada a consultas solo de estado.

## Resultado propuesto
Los clientes ven el estado del siniestro, el siguiente paso y la
fecha estimada en el portal.

## Usuarios y sistemas afectados
Personal de siniestros, equipo de portal, API de claims-core.

## Restricciones
Ningún dato personal (PII) nuevo en la sesión del portal.
Solo autenticación existente.

## Preguntas abiertas
¿También necesitan acceso los peritos externos?
```

CONSIDERACIONES DE GOBERNANZA

La evidencia es el `intent.md` confirmado, que incluye autoría, marca de tiempo e historial completo de revisiones, registrado en el historial de git del repositorio de intención. El product owner aprueba, y la decisión de aceptar o rechazar que envía la intención a la Etapa 2: Design queda registrada como el merge o el cierre de la revisión.

CÓMO MEDIRLO

INDICADOR ADELANTADO

Tiempo desde la primera conversación hasta un `intent.md` confirmado, leído del historial de git del repositorio de intención, que registra autoría y marca de tiempo. Se espera que caiga de un ciclo de elicitación y refinamiento de varias semanas a solo horas.

INDICADOR REZAGADO

La tasa de supervivencia: el porcentaje de archivos `intent.md` que el product owner acepta hacia la Etapa 2: Design frente a los que cierra. También el número de cambios hechos al `intent.md` después del primer commit de `spec.md` para el mismo cambio.

02 Design

Requisitos y diseño se fusionan en una sola sesión. La política se aplica mientras se escribe el spec, no se descubre en una revisión semanas después.

Requisitos y diseño

Una vez aprobado por el product owner, Claude toma el `intent.md` aceptado y produce un spec de requisitos y diseño, guiado por las skills de la organización para marca, seguridad, cumplimiento y UX.

El product owner revisa ese spec, pero no lo escribe. El objetivo de este proceso es crear un spec sobre el que el equipo de ingeniería pueda planificar, con las áreas de preocupación señaladas.

El trabajo de frontend es el ejemplo más claro. Una vez aceptado el `intent.md`, el product owner maqueta el diseño en Claude Design (beta) a partir del `intent.md`, itera sobre la maqueta, y luego la exporta a Claude Code para construirla.

Tradicional

Requisitos y diseño son fases separadas a cargo de equipos separados. Los analistas formalizan la idea en requisitos, y luego los diseñadores los reinterpretan como diseño. La separación existe por responsabilidad, pero es lenta y pierde información.

Nativo de IA

Ambas fases ocurren en una sola sesión con prompts. Claude toma el `intent.md` y produce un spec de requisitos y diseño, restringido por las skills de la organización, con las áreas de preocupación señaladas.

CÓMO EMPEZAR

REQUISITOS PREVIOS

Un archivo `intent.md` redactado, con las políticas de marca, seguridad, cumplimiento y UX escritas como skills.

INFRAESTRUCTURA

Un product owner con acceso a Claude. No se requiere ninguna habilidad de ingeniería.

CÓMO EJECUTARLO

1. El product owner abre una sesión con las skills de la organización disponibles y adjunta el `intent.md`.
2. El prompt del product owner apunta al `intent.md`, nombra las restricciones, y exige que se señalen las preocupaciones. Se ejecuta a mano al principio, y luego se codifica como un slash command a nivel de organización. A partir de ahí, la aceptación del `intent.md` en su repositorio se convierte en el disparador: un job no interactivo se activa con el merge, ejecuta el pase con las skills de la organización cargadas, y confirma el `spec.md` como un pull request (el play de CI/CD en la Etapa 5: Deploy cubre esa fontanería). Desde ese punto, la primera intervención del product owner es la revisión.
3. El mismo product owner revisa el spec contra la idea original. ¿El spec resuelve el problema planteado? ¿Las preguntas abiertas del `intent.md` quedan respondidas o se trasladan?
4. Se trabajan primero las preocupaciones señaladas, porque son los puntos que un analista habría escalado. El product owner resuelve cada una con su responsable de política antes de que ingeniería vea el spec.
5. Se confirma el `spec.md` junto al `intent.md`. El par de archivos registra qué se pidió y qué se decidió.
6. El product owner decide si el spec y la intención avanzan a construcción, consultando a un lead técnico para todo lo que la organización clasifique como de mayor riesgo. Esta decisión siempre la toma una persona del equipo, y aceptar el spec es lo que da inicio al play del modo plan en la Etapa 3: Build.

Así se ve (el prompt)

```
Lee el intent.md adjunto y produce un spec de requisitos y diseño
para integrarlo en nuestra base de código existente. Aplica las
skills disponibles para que el plan cumpla con nuestras guías de
marca, políticas de seguridad y estándares de UX. Documenta el spec
por completo como spec.md, listo para entregar al equipo de
ingeniería. Describe con claridad cualquier área de preocupación,
especialmente donde no puedas satisfacer políticas contradictorias.
```

CONSIDERACIONES DE GOBERNANZA

En lugar de descubrirse en una revisión semanas después, la política vigente se lee y se aplica mientras se escribe el spec. Las skills de la organización se aplican como restricciones sobre el spec. El spec, el prompt que lo produjo, y las versiones de las skills vigentes quedan registrados en el control de versiones. El product owner firma el spec y enruta las preocupaciones señaladas a los responsables de política correspondientes.

CÓMO MEDIRLO

INDICADOR ADELANTADO

Tiempo transcurrido entre el commit del `intent.md` y el del `spec.md` para el mismo cambio (dos marcas de tiempo en git), comparado con el antiguo ciclo de requisitos más diseño.

INDICADOR REZAGADO

Retrabajo de requisitos después de empezar la construcción. Se cuentan los commits de `spec.md` posteriores al primer commit de `plan.md` para el mismo cambio; el log de git lo da directamente.

03 Build

Nada se implementa sin un plan aceptado. El conocimiento institucional se convierte en archivos que el agente lee, y las barreras de protección se ejecutan como código en lugar de como hábitos.

El modo plan de Claude Code como punto de partida por defecto

Las personas ingenieras empiezan las sesiones de Claude Code en modo plan, le dan a Claude el `spec.md` aprobado en la Etapa 2: Design, y dejan que las entreviste, iterando sobre el plan hasta que la persona ingeniera esté conforme con él.

Tradicional

Una persona ingeniera lee el diseño y empieza a escribir código. Cómo se hará el cambio, hasta qué archivos y qué pruebas, queda en su cabeza o, en el mejor de los casos, en un comentario de un ticket. Nadie más puede revisarlo. Lo primero que ve quien revisa es el diff terminado, y para entonces el retrabajo es lento.

Nativo de IA

El trabajo empieza con un plan escrito que Claude produce en modo plan, donde puede leer la base de código sin cambiar nada. La persona ingeniera corrige el plan antes de que se escriba código, y la versión aprobada se confirma como `plan.md` para que etapas posteriores lo verifiquen.

CÓMO EMPEZAR

REQUISITOS PREVIOS

El artefacto de intención (`intent.md` o `spec.md`) si existe; el archivo `CLAUDE.md` ayuda.

INFRAESTRUCTURA

Claude Code con acceso al repositorio.

CÓMO EJECUTARLO

1. La persona ingeniera empieza la sesión en modo plan con Claude.
2. Le da a Claude el `intent.md` y el `spec.md` y le pide un plan de implementación que nombre los archivos que cambian, el orden del trabajo y las pruebas que lo demuestran.
3. Interroga el plan preguntando qué podría romper el cambio, qué paso es más arriesgado, y qué otras opciones descartó Claude.
4. Itera hasta que una persona que nunca haya visto la conversación pudiera implementar el cambio solo con el plan.
5. Confirma el plan aprobado como `plan.md`. El plan se suma al rastro de auditoría, y el play de revisión de PR (Etapa 5: Deploy) verifica el diff final contra él.
6. Acepta el plan y deja que Claude implemente. Con un buen plan, la implementación suele resolverse en una sola pasada.
7. Cuando la implementación se desvía del plan, actualiza el `plan.md` en el mismo commit. Puede usarse un hook para forzar la sincronización entre ambos.

Así se ve (plan.md)

```
# Plan: autoservicio de estado de siniestros (desde intent.md 2026-06-02)

## Archivos que cambian
portal/src/claims/StatusPanel.tsx (nuevo), claims-api/routes/status.py,
claims-api/tests/test_status.py

## Orden de trabajo
1. Añadir el endpoint de estado detrás de la autenticación existente.
2. Construir el panel contra el endpoint.
3. Integrarlo en la navegación del portal.

## Riesgos
La API de claims-core limita a 50 rps; el panel debe cachear.

## Prueba
test_status.py cubre los cuatro estados del siniestro; la captura
de pantalla coincide con la maqueta aprobada.
```

CONSIDERACIONES DE GOBERNANZA

La revisión de diseño ocurre antes de generar cualquier código, cuando cambiar de rumbo sigue siendo cuestión de editar un documento. El propio modo plan lo hace cumplir, ya que Claude no puede editar archivos hasta que la persona ingeniera acepta el plan. El plan y sus revisiones quedan registrados junto con quién lo aceptó. Los cambios rutinarios los aprueba la persona ingeniera, y todo lo que la organización clasifique como mayor riesgo pasa a un lead técnico o arquitecto.

CÓMO MEDIRLO

INDICADOR ADELANTADO

Porcentaje de cambios que se fusionan desde la primera pasada de implementación, y tiempo desde la aprobación del plan hasta el PR fusionado, con los datos necesarios en los metadatos del PR.

INDICADOR REZAGADO

Ciclos de retrabajo por cambio, también desde los metadatos del PR, y con qué frecuencia el diff fusionado sigue coincidiendo con el `plan.md` confirmado.

Claude Code en modo automático

Claude Code también puede ejecutarse en modo automático (auto mode), donde la persona ingeniera aprueba el plan y, una vez conforme e iterado, Claude aplica cada cambio sin pedir confirmación edición

por edición. A medida que maduran las barreras de protección de los siguientes plays —un `CLAUDE.md` afinado, skills que codifican políticas, hooks que bloquean acciones inseguras, y una suite de pruebas que Claude puede ejecutar—, la aceptación automática se convierte en el modo por defecto para el trabajo rutinario: un `spec.md` ajustado, un radio de impacto pequeño, y código ya cubierto por pruebas.

El cambio ahora va de que la persona observe al agente hacer cada edición y revise cada acción, hacia la revisión de artefactos después de sesiones autónomas más largas. El modo de aceptación automática además habilita el paralelismo entre personas y equipos cuando se usa con worktrees, y es fundamental para ejecutar el SDLC de forma autónoma y cerrar el bucle, como se describe en la Etapa 6: Maintenance.

RECUADRO

SISTEMAS LEGACY Y LA FUENTE DE VERDAD

Aplica a todos los artefactos que produce el proceso.

Los procesos de SDLC existentes probablemente ya rastrean artefactos, solo que no en archivos markdown. Los work items pueden estar en Jira, los requisitos en una herramienta con trazabilidad regulatoria incorporada, los diseños en Figma, y las aprobaciones de cambio en un comité de cambios. Esos sistemas son difíciles de desplazar porque auditores y reguladores ya los aceptan y otros equipos dependen de ellos, así que el SDLC nativo de IA tiene que encajar alrededor de lo que ya existe.

Al transicionar hacia el SDLC nativo de IA, para cada artefacto que produce el proceso hay que nombrar un sistema como fuente de verdad, y todo lo demás guarda una copia o un enlace al original. Las siguientes configuraciones tienen una única fuente de verdad, con la elección variando según el artefacto:

- **El repositorio como fuente de verdad.** Los artefactos markdown son el registro autoritativo y el sistema legacy referencia archivos dentro de commits. Puede ser una de las configuraciones más limpias para organizaciones lideradas por ingeniería, ya que todos los registros viven en una sola herramienta con una sola autoridad de marca de tiempo.
- **El sistema legacy como fuente de verdad.** Jira, ServiceNow, o la herramienta de requisitos guarda el registro autoritativo, y los artefactos markdown son copias de trabajo. Claude lee el registro al inicio de la sesión y escribe el resultado de vuelta a través de un conector MCP en la misma sesión que produjo el spec o el plan.
- **El enlace como mínimo indispensable.** Todos los artefactos anotan el ID del registro, y todos los registros legacy contienen el SHA del commit del archivo markdown. Es un buen punto de partida al transicionar hacia el SDLC nativo de IA, aceptando que hay dos fuentes de verdad.

Tanto el sistema legacy como el sistema markdown-first pueden coexistir, siempre que haya un enlace entre ambos o se declare una fuente de verdad.

El CLAUDE.md

`CLAUDE.md` le da a Claude el contexto que necesitaría alguien que se incorpora al equipo: convenciones, comandos, arquitectura, y los errores que el equipo ve con más frecuencia. El

conocimiento que antes vivía en la cabeza de las personas y en wikis se convierte en un archivo que el agente lee al inicio de cada sesión, mantenido por todo el equipo e iterado cada vez que se comete un error.

CÓMO EMPEZAR

REQUISITOS PREVIOS

Ninguno.

INFRAESTRUCTURA

Un repositorio, Claude Code instalado, y una persona ingeniera que conozca bien la base de código.

CÓMO EJECUTARLO

1. Ejecuta `/init` en el repositorio. Claude genera un `CLAUDE.md` inicial a partir de lo que encuentra.
2. Recorta el archivo generado a lo que necesitaría alguien nuevo el primer día. Conserva los comandos de build, test y lint, las convenciones que importan, y las cosas que Claude sigue haciendo mal.
3. Sube el `CLAUDE.md` a git en la raíz del repositorio para que todo el equipo comparta una sola versión y los cambios se revisen como código.
4. Una regla práctica ayuda aquí: cuando Claude comete el mismo error dos veces, la corrección va al `CLAUDE.md`.
5. Mantenlo por debajo de una página, porque Claude lo lee entero al inicio de cada sesión y todo lo que esté obsoleto ocupa contexto sin ningún beneficio.

Así se ve (CLAUDE.md)

```
# Servicio de pagos

## Comandos
- Build: make build
- Test: make test (unitarias), make itest (integración, necesita docker)
- Lint: make lint (se ejecuta en CI; corrígelo antes de hacer push)

## Convenciones
- Java 21, Spring Boot 3. Nada de Lombok nuevo.
- El dinero siempre es BigDecimal, nunca double.
- Todo endpoint necesita una prueba de integración en src/itest.

## Arquitectura
- api/ contiene los controladores REST, core/ la lógica de dominio,
  adapters/ se comunica con sistemas externos.
- Los eventos de Kafka se definen en schemas/; nunca edites las
  clases generadas.

## Cosas que Claude hace mal
- No subir versiones de dependencias; el equipo de plataforma es
  quien las gestiona.
- El paquete legacy v1/ está congelado; los cambios van en v2/.
```

CONSIDERACIONES DE GOBERNANZA

`CLAUDE.md` está bajo control de versiones, así que las instrucciones con las que trabaja el agente son revisables y auditables. Las convenciones del equipo se aplican a través del archivo, sus cambios quedan registrados en el historial de git, y quienes son dueños del código aprueban esos cambios en la revisión del PR.

CÓMO MEDIRLO

INDICADOR ADELANTADO

Con qué frecuencia Claude repite un error que el `CLAUDE.md` debería haber evitado. Las correcciones o cambios al archivo deberían rastrearse en el historial de git.

INDICADOR REZAGADO

Tiempo hasta el primer PR fusionado de una persona nueva en el equipo, desde el historial de PRs.

Skills como conocimiento institucional

Las skills son la forma en que una organización hace operativo su conocimiento institucional. Las instrucciones son explícitas, están versionadas, se aplican de forma amplia, y se actualizan de manera centralizada cuando cambia la política. La regla general: escribe una skill para el conocimiento institucional que deba aplicarse de forma consistente; no escribas una skill para componentes que pertenecen al `CLAUDE.md` o a un prompt.

CÓMO EMPEZAR

REQUISITOS PREVIOS

Ninguno estrictamente necesario. Tener un `CLAUDE.md` ayuda, porque mantiene el conocimiento de trabajo del agente en el repositorio, pero una skill no depende de él.

INFRAESTRUCTURA

Una política con un responsable con nombre y una fuente de verdad escrita.

CÓMO EJECUTARLO

1. Elige una pieza de conocimiento que hoy se aplique de forma inconsistente. Puede ser un estándar de seguridad, una convención de diseño de API, o una regla de marca.
2. Escríbela como skill: una carpeta que contiene un `SKILL.md` cuyo frontmatter indica cuándo se activa y cuyo cuerpo indica qué hacer. Una persona ingeniera la redacta a partir de la fuente de verdad de quien es responsable de la política, con ayuda de Claude.
3. Coloca la skill en el repositorio en `.claude/skills/<nombre>/` para que viaje con el código, o distribúyela en toda la organización a través de un plugin.
4. Comprueba que la skill se activa. Pide a Claude que haga la tarea correspondiente de distintas formas y confirma que la skill se carga cada vez.
5. Cuando cambie la política, cambia la skill y haz que quien es responsable de la política la firme.
6. Las personas ingenieras reciben la nueva versión automáticamente en su siguiente sesión.

Así se ve (.claude/skills/secure-api-review/SKILL.md)

```
---
name: secure-api-review
description: Apply the API security standard. Use whenever creating or
  modifying an external-facing endpoint, reviewing API code, or
  generating an OpenAPI spec.
---

# Revisión de seguridad de API

Cuando crees o cambies un endpoint de API:

1. Autenticación: todo endpoint requiere el JWT del gateway;
   ninguna ruta anónima fuera de /health.
2. Validación de entrada: valida los cuerpos de las peticiones
   contra el schema de OpenAPI y rechaza campos desconocidos.
3. Auditoría: todo endpoint que cambia estado emite un evento de
   auditoría con actor, acción, entidad y marca de tiempo.
4. Clasificación de datos: los campos etiquetados como pii en el
   schema nunca deben aparecer en logs ni en mensajes de error.

Ejecuta scripts/check-endpoints.sh e incluye su salida en tu resumen.
```

CONSIDERACIONES DE GOBERNANZA

Una skill es un control, aunque de carácter orientativo. Hace que sea probable que Claude aplique la política mientras se escribe el código, pero nada obliga a una sesión a cumplirla. Una política que siempre debe cumplirse necesita algo determinista detrás de la skill, como un hook que bloquee la acción o un pase de revisión que vuelva a comprobar la política en el PR. La skill hace que las infracciones sean raras, y el hook las hace casi imposibles. Las invocaciones de skills quedan registradas en las trazas de sesión, y quien es responsable de la política revisa sus cambios como si fuera código.

CÓMO MEDIRLO

INDICADOR ADELANTADO

Tiempo desde que el responsable de la política aprueba un cambio hasta que la skill actualizada se fusiona, tomado del PR sobre la carpeta de la skill.

INDICADOR REZAGADO

Hallazgos de revisión de PR que citan la política, que deberían tender a cero una vez que la skill aplica la política mientras se escribe el código. Si no caen, la skill no se está activando o su texto se ha desviado de la política oficial.

Hooks como barreras de protección en tiempo de construcción

Una skill es un control orientativo, mientras que un hook es la capa determinista que hay detrás. La mayoría de las acciones de Claude son ediciones de archivos y comandos de shell durante la implementación, así que la fase de construcción es donde los hooks suelen dispararse con más frecuencia.

Los hooks en fase de construcción pueden:

- Bloquear ediciones a rutas protegidas, como clases generadas o un paquete congelado;
- Ejecutar el formateador y el linter después de las ediciones de archivos, para que la desviación nunca se acumule;
- Mantener las credenciales fuera del diff.

Respaldan cualquier skill cuya política tenga que cumplirse sin excepción. Un hook se ejecuta en cada acción que coincida con él, así que los hooks de fase de construcción deben ser rápidos y limitarse al archivo que cambió. Las comprobaciones más pesadas, como la suite de pruebas completa, pertenecen al commit o al PR.

Un hook que le pide aprobación a una persona pertenece a las puertas de la Etapa 5: Deploy, porque un aviso de aprobación durante la construcción pone a una persona de vuelta en la ruta crítica de todas las sesiones que corren en paralelo.

Sesiones paralelas y subagentes

Una sola persona ingeniera puede llevar varias líneas de trabajo a la vez.

Una sesión paralela es otra instancia completa de Claude Code, trabajando en una tarea separada dentro de su propio worktree de git. Cada sesión independiente no sabe nada de las demás, y lo único que comparten es la persona que las dirige.

Un subagente se ejecuta dentro de una sola sesión como ayudante con alcance limitado, con su propia ventana de contexto y sus propios límites de herramientas, y es útil para tareas que se repiten en varias tareas, como verificar que la aplicación funciona como se espera.

Las sesiones paralelas aumentan el número de tareas que una persona ingeniera puede tener en marcha a la vez, mientras que los subagentes mantienen cada sesión enfocada en su propia tarea. El trabajo de la persona ingeniera es dirigir y revisar todas ellas.

Tradicional

Una persona ingeniera trabaja en una tarea a la vez y dedica buena parte de su día o semana a builds, pruebas y revisiones. Cambiar de tarea mientras espera es posible, pero el cambio de contexto cansa lo suficiente como para que pocas personas lo elijan.

Nativo de IA

Una persona ingeniera ejecuta varias sesiones de Claude a la vez, cada una en su propio worktree y su propia tarea. Los trabajos repetidos se convierten en subagentes con su propio contexto y límites de herramientas. El trabajo de la persona ingeniera pasa a ser orquestar, y con el tiempo, a construir y supervisar bucles.

CÓMO EMPEZAR

REQUISITOS PREVIOS

El `CLAUDE.md`, ya que todas las sesiones lo leen. El bucle de retroalimentación (Etapa 4: Test) también ayuda aquí, porque una sesión que puede verificar su propio trabajo necesita menos supervisión de la persona ingeniera.

INFRAESTRUCTURA

Un repositorio git, ya que el aislamiento viene de los worktrees, y ajustes de permisos calibrados para que las sesiones no queden esperando prompts de aprobación en comandos que la organización considera seguros.

CÓMO EJECUTARLO

1. La persona ingeniera divide el trabajo en tareas que tocan archivos distintos, usando el plan del play de modo plan (Etapa 3: Build) para ver dónde el trabajo es independiente. Las tareas que comparten archivos se ejecutan en una sola sesión, una tras otra.
2. Cada tarea paralela recibe su propio worktree, por ejemplo `claude --worktree feature-auth` en una terminal y `claude --worktree fix-rate-limit` en otra. Un worktree es un checkout separado en su propia rama, lo que evita que las sesiones choquen sobre los mismos archivos.
3. Dos o tres sesiones es un buen punto de partida. El techo práctico es cuántas líneas de trabajo puede revisar bien una sola persona, así que se añaden sesiones solo mientras la revisión pueda seguir el ritmo.
4. Convierte los trabajos repetidos en subagentes, definidos en archivos markdown dentro de `.claude/agents/`, cada uno con un nombre, una descripción de cuándo usarlo, y las herramientas que puede tocar. Algunos ejemplos: un simplificador de código que elimina complejidad innecesaria al terminar el agente principal, un verificador que ejecuta la aplicación y comprueba el comportamiento, o un investigador que explora la base de código e informa sin saturar el contexto principal. Se suben a git para que todo el equipo los comparta.

Así se ve (.claude/agents/verifier.md)

```
---
name: verifier
description: Runs the app and checks the change works before the session
  reports done
tools: Bash, Read
---

Arranca la aplicación con make run. Ejercita el comportamiento
cambiado y los dos flujos vecinos más cercanos. Informa qué
ejecutaste, qué observaste, y cualquier comportamiento que no
coincida con plan.md. No arregles nada; solo informa.
```

CONSIDERACIONES DE GOBERNANZA

Más sesiones significa más resultados, así que los controles tienen que venir de la configuración en el repositorio. Los hooks y los ajustes de permisos ahí definidos se aplican a todas las sesiones, y lo que hace una sesión queda registrado y atribuido a la persona ingeniera que la ejecutó.

CÓMO MEDIRLO

INDICADOR ADELANTADO

Sesiones concurrentes por persona ingeniera mientras se mantiene la calidad de revisión, contadas desde la exportación de OpenTelemetry, y el porcentaje del día dedicado a dirigir en lugar de esperar.

INDICADOR REZAGADO

Cambios fusionados por persona ingeniera por semana, leído junto con la tasa de retrabajo según el historial de PRs.

04 Test

Cada sesión revisa su propio trabajo antes de que lo vea una persona, y la configuración que dirige al agente se somete a pruebas de regresión igual que el código que escribe.

Dale a Claude un bucle de retroalimentación

Dale siempre a Claude una forma de verificar su propio trabajo, ya sean pruebas, un build, o una comparación de capturas de pantalla. Una sesión revisa su propio trabajo y corrige sus propios errores antes de que una persona ingeniera los vea.

El bucle de retroalimentación no debe confundirse con un subagente verificador (Etapa 3: Build). El bucle de retroalimentación se ejecuta a lo largo de toda la tarea, tantas veces como haga falta. El subagente verificador, en cambio, es una forma de empaquetar la comprobación final ejecutando una ventana de contexto nueva una vez que la sesión cree que el trabajo está terminado. Así, el veredicto no queda contaminado por las suposiciones que produjeron el código.

Tradicional

La señal de que el código funciona llega tarde: CI minutos después, alguien de testing días después, producción semanas después. Con un agente produciendo el código, una señal tardía significa que una persona tiene que revisar todo su resultado, y esa persona se convierte en el cuello de botella.

Nativo de IA

A la sesión se le da una forma de revisar su propio trabajo antes de que lo vea una persona. Ejecuta las pruebas, ejecuta el build, toma la captura de pantalla. Claude itera hasta que la comprobación pasa, así que lo que llega a la persona ingeniera ya la ha superado. Configurar el bucle recae en quien dirige la sesión, y los pasos siguientes están escritos para esa persona.

CÓMO EMPEZAR

REQUISITOS PREVIOS

Ninguno.

INFRAESTRUCTURA

Una suite de pruebas y un build que se ejecuten localmente con un solo comando cada uno. Para trabajo de interfaz, es crucial que Claude pueda ver el resultado, ya sea con una herramienta de navegador o una utilidad de capturas conectada vía MCP.

CÓMO EJECUTARLO

1. Si hoy verificar el trabajo requiere una secuencia de comandos y algo de conocimiento del entorno, envuélvelo en un único target como `make test` o `npm test` que termine con código distinto de cero si falla.
2. En la sección de Comandos del `CLAUDE.md`, lista cada comando con un ejemplo de salida saludable.
3. Define un objetivo y hazlo cuantificable para que Claude pueda verificar el trabajo sin preguntarte, por ejemplo: «todas las pruebas en test_status.py pasan», «la captura coincide con la maqueta adjunta», o «el endpoint devuelve 200 con el campo nuevo».
4. Para corregir errores, escribe primero la prueba que falla. Pide a Claude que reproduzca el error como una prueba, la ejecute, y confirme que falla por la razón esperada. Confirma esa prueba. Solo entonces pide a Claude que la haga pasar sin editar la prueba, con el hook del último paso reforzando esa restricción sobre archivos de test. Una prueba que existía antes de la corrección, y que el agente no pudo reescribir, es la prueba de que el error desapareció.
5. Para trabajo de interfaz, cierra el bucle con una comprobación visual. Dale a Claude una herramienta de navegador o de capturas, dale la maqueta, y deja que itere: implementa, captura, compara, ajusta. Dos o tres rondas es normal, y el resultado debería mejorar en cada una.
6. Haz que la verificación forme parte de «terminado». La instrucción vive en el `CLAUDE.md`: ejecutar las pruebas antes de reportar una tarea como completa, y mostrar la salida.
7. Por último, el propio bucle necesita protección, porque un agente que corrige código no debe poder debilitar la comprobación sobre ese código. Un hook que bloquee ediciones a archivos de prueba durante una tarea de corrección logra esto. La alternativa es revisar el diff y rechazar cualquier cambio que toque una prueba.

Así se ve (bloque de verificación en CLAUDE.md)

```
## Verificando tu trabajo

- Build: make build (debe terminar con "Build succeeded")
- Test: make test (todo en verde; nunca saltes ni borres una prueba que falla
- Lint: make lint (cero advertencias)

Ejecuta los tres antes de reportar cualquier tarea como completa, y
pega la salida.

Si una prueba falla, corrige el código, no la prueba.
```

CONSIDERACIONES DE GOBERNANZA

QUÉ SE APLICA	La verificación antes de reportar una tarea como terminada, y el bloqueo sobre la edición de archivos de prueba durante una corrección, ambos implementados como hooks donde la organización quiere garantías.
CUÁL ES LA EVIDENCIA	La salida literal de «make test», el log del build, o la comparación de capturas que Claude ejecutó y pegó, así que la evidencia proviene de la propia cadena de herramientas.
DÓNDE SE REGISTRA	En la transcripción de la sesión, que la exportación de OpenTelemetry reenvía al stack de observabilidad de la organización, y en el check run del PR, donde tanto quien revisa como cualquier auditoría posterior pueden verla.
QUIÉN APRUEBA	La persona dueña del código que revisa el PR, que puede concentrarse en la intención y el riesgo porque la evidencia mecánica ya está adjunta.

CÓMO MEDIRLO

INDICADOR ADELANTADO

Tasa de éxito en la primera pasada de CI para cambios escritos por el agente, que el propio sistema de CI ya soporta.

INDICADOR REZAGADO

Tiempo de revisión por PR (de los metadatos del PR), que debería caer una vez que las pruebas atrapan lo que antes atrapaban quienes revisaban, y la tasa de fallos por cambio desde un rastreador de incidentes.

Evaluaciones continuas en CI

Las evaluaciones (evals) son el equivalente nativo de IA al control de calidad por etapas. En la práctica, eso significa una suite que se ejecuta cada vez que cambia la configuración del agente. Cuando se

cambia de modelo o se reescribe un prompt, la suite de evals dice si el agente sigue haciendo el trabajo con el mismo estándar.

Las evals deben verse como una suite viva. A medida que los modelos mejoran, los casos que antes discriminaban dejan de hacerlo, y hay que añadir otros nuevos que surgen del monitoreo continuo.

Según el caso de uso, algunos equipos pueden preferir ejecutar estas evaluaciones fuera de línea con una cadencia fija en lugar de en cada cambio. Los pasos siguientes son para evaluaciones continuas.

CÓMO EMPEZAR

REQUISITOS PREVIOS

El `CLAUDE.md` y el bucle de retroalimentación (Etapa 4: Test).

INFRAESTRUCTURA

Un CI que pueda ejecutar Claude Code de forma no interactiva, y una API key con presupuesto para las ejecuciones de evals.

CÓMO EJECUTARLO

1. La persona de plataforma reúne entre 20 y 50 tareas reales de trabajo reciente, con su resultado esperado o aceptado.
2. Cada tarea se escribe como un eval: el prompt más las comprobaciones que definen lo aceptable (las pruebas pasan, el lint está limpio, el comportamiento no cambió, se siguió la política).
3. La suite se ejecuta de forma no interactiva en CI, según un horario y ante cualquier cambio al `CLAUDE.md`, las skills o los hooks, ya que esa configuración dirige al agente y merece las mismas pruebas de regresión que el código.
4. Los cambios de configuración se condicionan a los resultados. Un cambio de skill que baja la tasa de éxito se revisa antes de fusionarse.
5. Cada incidente de producción recibe un eval, escrito por el equipo dueño del incidente, y permanece en la suite como prueba de regresión.

Así se ve (.github/workflows/agent-evals.yml)

```yaml
name: Agent evals
on:
  pull_request:
    paths: ['CLAUDE.md', '.claude/**']
  schedule:
    - cron: '0 2 * * *'
jobs:
  evals:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install -g @anthropic-ai/claude-code
      - name: Run eval suite
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
        run: |
          for eval in evals/*.json; do
            claude -p "$(jq -r '.prompt' $eval)" \
              --allowedTools "Read,Edit,Bash(make test)" \
              --output-format json > result.json
            ./evals/check.sh "$eval" result.json
          done
```

CONSIDERACIONES DE GOBERNANZA

Las evals le dan al control de calidad una puerta que puede seguir el ritmo del agente. El umbral de tasa de éxito se aplica como comprobación de merge, las ejecuciones quedan registradas para comparar resultados a lo largo del tiempo, y el equipo dueño del cambio de configuración lo aprueba.

CÓMO MEDIRLO

INDICADOR ADELANTADO

La tasa de éxito de las evals a lo largo del tiempo, reportada por la suite en cada ejecución, y cuánto tarda un incidente de producción en convertirse en un eval permanente.

INDICADOR REZAGADO

Regresiones capturadas en CI comparadas con las encontradas en producción, según el rastreador de incidentes.

05 Deploy

La revisión funciona en ambas direcciones, y la gobernanza se aplica mientras el agente actúa. El agente hace todo hasta la puerta de producción, y nada después de ella.

La IA en el ciclo de revisión de PR

Claude tanto da como recibe revisiones. Revisa los PR entrantes contra las políticas de la organización, y atiende los comentarios de revisión sobre sus propios PR. Esto permite que las personas ingenieras se concentren, en su revisión, en el comportamiento, que en el fondo se reduce a juzgar intención y riesgo.

Tradicional

La capacidad de revisión se planificaba en torno al rendimiento humano. Un PR espera a que alguien lo lea entero, la calidad de la revisión varía según la carga de esa persona, y quien lo escribió tiene que perseguir mientras el backlog crece.

Nativo de IA

Todos los PR reciben el mismo conjunto de pases de revisión, con hallazgos clasificados por severidad. La atención humana sube de nivel: a si el cambio hace lo que el plan pretendía y si el riesgo es aceptable.

CÓMO EMPEZAR

REQUISITOS PREVIOS

Un `CLAUDE.md` actualizado desde la Etapa 3: Build; skills si los pases de revisión aplican políticas escritas; subagentes definidos.

INFRAESTRUCTURA

Un repositorio con la integración de Claude instalada, ya sea el servicio gestionado de Code Review (vista previa de investigación) habilitado por una persona admin, o el `claude-code-action` corriendo en tu propio CI, con llamadas al modelo a través de AWS Bedrock, Google Vertex o Microsoft Foundry cuando haga falta (el play de CI/CD cubre las opciones de despliegue). Las políticas de protección de rama que exigen aprobación de un code owner también son recomendables.

CÓMO EJECUTARLO

1. El servicio gestionado de Code Review es la forma más rápida de empezar: una persona admin lo habilita y selecciona repositorios. Ejecuta la revisión en tu propio CI con el `claude-code-action` cuando necesites control del pipeline o quieras que las llamadas a la API pasen por tu propio acuerdo de nube (el play de CI/CD cubre esa fontanería).
2. El lead técnico escribe la política de revisión como `REVIEW.md` en la raíz del repositorio, dividida en los pases que le importan a la organización: errores y fallos lógicos; seguridad y vulnerabilidades; cumplimiento contra el spec (`spec.md` del play de requisitos), el plan de implementación (`plan.md` del play de modo plan) y los principios de diseño. `REVIEW.md` también define qué cuenta como Importante frente a un Nit, y qué se omite.
3. El lead técnico fija el umbral humano. Los hallazgos no aprueban ni bloquean un PR por sí solos, y la protección de rama sigue exigiendo la aprobación de un code owner. Quien en plataforma quiera condicionar los merges a los hallazgos puede leer los recuentos de severidad que el check run publica como un resumen legible por máquina.
4. Cuando quien revisa o quien escribió etiqueta a `@claude` en un comentario de revisión, Claude atiende el comentario y sube la corrección. El hilo del PR registra tanto la petición como el cambio. Este bucle de corrección se ejecuta a través del `claude-code-action`. En el servicio gestionado, comentar `@claude review` solicita una revisión nueva en su lugar. Para los PR que abrió Claude, se puede ir más allá y dejar que Claude cuide el PR hasta el merge: algunos equipos envuelven el bucle en un slash command personalizado que recorre los comentarios de revisión sin resolver y las comprobaciones que fallan en el PR, los atiende y sube las correcciones, hasta que el PR está en verde y solo espera la aprobación del code owner.

5. Los hallazgos de revisión retroalimentan el `CLAUDE.md`. Cuando una revisión señala un error por segunda vez, la corrección va al `CLAUDE.md` como parte de esa revisión, y como la revisión lee el `CLAUDE.md`, el error se atrapa desde el siguiente PR en adelante. La revisión también señala cuándo un cambio ha dejado el `CLAUDE.md` desactualizado.
6. Una vez al mes, el lead técnico afina la configuración calificando los hallazgos para que quien revisa mejore, y limitando el volumen de Nits en `REVIEW.md`. Se excluyen las rutas generadas y todo lo que CI ya aplica.

Así se ve (REVIEW.md)

```
# Instrucciones de revisión

## Pases
Ejecuta tres pases y etiqueta cada hallazgo con su pase:
- Bugs: errores de lógica, casos límite rotos, regresiones sutiles
- Seguridad: riesgos de inyección, brechas de autenticación, PII en logs
- Cumplimiento: el cambio coincide con spec.md, plan.md y nuestros
  principios de diseño

## Qué significa "Importante" aquí
Reserva "Importante" para hallazgos que romperían el comportamiento,
filtrarían datos o incumplirían una política. El estilo y el
nombrado son nits.

## Limita los nits
Reporta como máximo cinco nits por revisión; resume el resto como
un conteo.

## No reportar
Archivos generados bajo src/gen/ y cualquier cosa que CI ya aplique.
```

CONSIDERACIONES DE GOBERNANZA

Se preserva la separación de funciones, porque el agente que escribió el código no tiene forma de aprobarlo. La política de revisión en `REVIEW.md` se aplica a todos los PR, y los hallazgos, correcciones, calificaciones y aprobaciones quedan registrados en el historial del PR, así que el PR es el registro de auditoría. La aprobación viene de una persona a través de la protección de rama, informada por los hallazgos.

Para ver cómo estos controles se combinan a escala de producción, consulta *securing an AI-native SDLC at Anthropic*.

CÓMO MEDIRLO

INDICADOR ADELANTADO	INDICADOR REZAGADO

Tiempo hasta la primera revisión, que debería caer a minutos, y el porcentaje de comentarios de revisión resueltos sin que una persona toque la rama, con datos guardados directamente en Git.

Defectos y vulnerabilidades capturados antes del merge frente a los que llegan a producción, desde el historial de PRs y el rastreador de incidentes.

Hooks como puertas de aprobación

La fase de construcción usaba los hooks como barreras de protección, permitiendo o bloqueando acciones sin involucrar a ninguna persona (Etapa 3: Build). Un hook también puede preguntar, pausando la acción hasta que una persona específica la apruebe, que es justo lo que necesita la puerta de release.

Este play se ubica en la Etapa 5: Deploy porque la puerta de release es el caso más claro, pero los hooks no son exclusivos del despliegue: se ejecutan dondequiera que actúe Claude. Por ejemplo, los hooks pueden bloquear ediciones a migraciones e infraestructura sin un ticket de cambio durante la Etapa 3: Build, y detener al agente cuando intenta editar archivos de prueba durante una tarea de corrección en la Etapa 4: Test.

CÓMO EMPEZAR

REQUISITOS PREVIOS

Ninguno.

INFRAESTRUCTURA

Una lista escrita de las aprobaciones que exige el proceso de cambio.

CÓMO EJECUTARLO

1. La dirección de ingeniería, junto con gestión del cambio y cumplimiento, enumera las puertas de aprobación humana que deben mantenerse, como la firma de gestión del cambio, la autorización de release, y las ediciones a rutas protegidas.
2. La persona de plataforma expresa cada puerta como un hook: un script que se ejecuta antes de que Claude actúe y que puede permitir, preguntar o bloquear.
3. Los hooks de equipo van en `.claude/settings.json` dentro de git, y los hooks innegociables van en la configuración gestionada, propiedad de plataforma o de IT, donde ninguna persona ingeniera puede desactivarlos.
4. Un bloqueo debe explicarse a sí mismo: cuando un hook detiene una acción, la razón y la ruta hacia la aprobación aparecen en la salida de Claude.

Así se ve (.claude/settings.json)

```
{
    "hooks": {
      "PreToolUse": [
        {
          "matcher": "Bash",
          "hooks": [
            { "type": "command",
              "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/production-gate
          ]
        }
      ]
    }
}
```

Y la puerta en sí (.claude/hooks/production-gate.sh)

```
#!/bin/bash
# Production deploys require a named release authorization
cmd=$(jq -r '.tool_input.command' < /dev/stdin)
if [[ "$cmd" == *"deploy"* && "$cmd" == *"production"* ]]; then
   if [ -z "$RELEASE_APPROVAL" ]; then
     echo "Production deploys need a release authorization." >&2
     exit 2 # exit 2 blocks the action; the message goes to Claude
   fi
fi
exit 0
```

CONSIDERACIONES DE GOBERNANZA

Los hooks son las puertas de aprobación. La condición de la puerta se aplica siempre, para todos. Las decisiones de permitir o bloquear quedan registradas con una marca de tiempo. La puerta también define qué cuenta como aprobación, ya sea un ticket de cambio aprobado o la firma de la persona responsable de release.

EJEMPLO TRABAJADO

CONFIGURACIÓN GESTIONADA PARA UNA EMPRESA REGULADA

Desplegada por el equipo de plataforma vía MDM o la consola de administración; las personas ingenieras no pueden editarla ni anularla.

```
{
  "permissions": {
     "deny": [
        "Read(.env*)", "Read(./secrets/**)",
        "WebFetch", "Bash(curl *)", "Bash(wget *)"
     ],
     "allow": [
        "Bash(git *)", "Bash(make build)",
        "Bash(make test)", "Bash(make lint)"
     ],
     "disableBypassPermissionsMode": "disable"
  },
  "allowManagedPermissionRulesOnly": true,
  "sandbox": {
     "enabled": true,
     "failIfUnavailable": true,
     "allowUnsandboxedCommands": false,
     "network": { "allowedDomains": ["git.internal.example.com",
"registry.npmjs.org"] },
     "credentials": {
        "files": [
          { "path": "~/.ssh", "mode": "deny" },
          { "path": "~/.aws/credentials", "mode": "deny" }
        ],
        "envVars": [ { "name": "GITHUB_TOKEN", "mode": "deny" } ]
     }
  },
  "allowManagedHooksOnly": true,
  "disableSideloadFlags": true,
  "allowManagedMcpServersOnly": true,
  "strictKnownMarketplaces": [
     { "source": "github", "repo": "example-corp/approved-plugins" }
  ],
  "requiredMinimumVersion": "2.1.193"
}
```

QUÉ COMPRA CADA LÍNEA, EN TÉRMINOS DE CONTROL

- `permissions.deny` mantiene los secretos fuera del contexto del agente y bloquea la salida arbitraria a la red a través de herramientas; `permissions.allow` preaprueba el

bucle interno seguro para que la lista de bloqueo no se convierta en fatiga de prompts.

- `disableBypassPermissionsMode` junto con `allowManagedPermissionRulesOnly` significa que ninguna persona ingeniera, archivo de proyecto o flag de línea de comandos puede ampliar las reglas.
- `sandbox` cierra la brecha que los permisos no pueden cerrar. Un bloqueo a nivel de herramienta sobre `WebFetch` no impide que un comando de shell llegue a la red; la lista blanca de dominios a nivel de sistema operativo bloquea esa salida por completo.
- `failIfUnavailable` y `allowUnsandboxedCommands` convierten el sandbox en una puerta real: Claude Code se niega a arrancar cuando el sandbox no puede inicializarse, y un comando que falla dentro del sandbox no puede reintentarse fuera de él.
- `credentials` cierra la brecha que dejan abierta las reglas de bloqueo. `permissions.deny` gobierna las herramientas de archivo de Claude, pero un comando de shell en sandbox aún podría leer por defecto `~/.ssh` o `~/.aws/credentials`; este bloque niega esas lecturas y retira los secretos nombrados del entorno de cada comando en sandbox.
- `allowManagedHooksOnly` significa que las puertas de aprobación de este play son los únicos hooks que se ejecutan; nada local puede añadir ni reemplazarlos.
- `disableSideloadFlags` y `strictKnownMarketplaces` significan que toda skill, agente, hook y servidor MCP en la máquina de una persona ingeniera llegó a través del marketplace de plugins aprobado por la organización, nunca desde un directorio personal.
- `allowManagedMcpServersOnly` convierte la superficie de herramientas del agente en una lista blanca propiedad del equipo de plataforma.
- `requiredMinimumVersion` se niega a arrancar en una versión por debajo del mínimo aprobado, así que los controles quedan reforzados por una build que la organización realmente ha evaluado.

Toma lo anterior como un punto de partida para adaptar, no como una recomendación para copiar tal cual. Cada bloqueo compite con capacidad, y el equilibrio correcto depende de la clasificación de datos del repositorio. La referencia de settings documenta cada clave, incluidas las exclusivas de configuración gestionada: `code.claude.com/docs/en/settings`

CÓMO MEDIRLO (PARA LOS HOOKS EN SÍ)

INDICADOR ADELANTADO

Tiempo de espera en cada puerta de aprobación. Cada decisión de hook se escribe

INDICADOR REZAGADO

Infracciones de puerta que llegan a producción antes y después de los hooks, según el

en la exportación de OpenTelemetry con una marca de tiempo y un veredicto de permitir o bloquear, así que la espera es visible por puerta.

rastreador de incidentes.

Integración y despliegue con CI/CD

Ejecuta Claude Code de forma no interactiva dentro del pipeline de CI/CD, aísla la ejecución en un sandbox para que los agentes de larga duración corran con seguridad, expón el despliegue a través de integraciones MCP, y ensaya las rutas de rollback antes de que el agente las necesite de verdad.

Tradicional

Los pipelines ejecutan scripts deterministas, y cualquier cosa que requiera criterio espera a una persona: por ejemplo, triar una prueba inestable, redactar el changelog, o averiguar por qué se rompió el build. El despliegue y el rollback son runbooks que una persona sigue bajo presión.

Nativo de IA

Claude se ejecuta de forma no interactiva dentro del pipeline para los pasos que requieren criterio, en un sandbox con credenciales limitadas. Las herramientas de despliegue se exponen al agente a través de MCP, así que el mismo flujo que escribió y probó el cambio también puede publicarlo y revertirlo, dentro de las puertas que la organización define por entorno.

CÓMO EMPEZAR

REQUISITOS PREVIOS

Claude en el ciclo de revisión de PR y los hooks como puertas de aprobación, porque las puertas deben existir antes de que la automatización acelere algo a través de ellas.

INFRAESTRUCTURA

Una plataforma de CI con el `claude-code-action` instalado, o cualquier runner que pueda invocar `claude -p`; acceso al modelo vía API, o Bedrock, Foundry o Vertex cuando el tráfico deba quedarse dentro del acuerdo de nube de la organización; servidores MCP para los destinos de despliegue; un perfil de sandbox para los jobs del agente sin credenciales de producción permanentes.

CÓMO EJECUTARLO

1. La persona de plataforma empieza con pasos de criterio de solo lectura. Usa `claude -p` en un job del pipeline para triar un build fallido, resumir una prueba inestable, o redactar el changelog.
2. Añade pasos de escritura detrás de las puertas ya existentes, para tareas como arreglar el lint, actualizar documentación generada, o atender comentarios de revisión vía menciones a `@claude`. Todo lo que el agente escribe llega como un PR a través de la protección de rama, y el agente no tiene ninguna ruta directa a `main`.
3. La ejecución va en sandbox. Los jobs del agente corren en contenedores bajo una política de red con tokens de corta duración y alcance limitado, y no tienen credenciales de producción por defecto.
4. Expón el despliegue a través de MCP. Deploy, status y rollback se convierten en herramientas, con alcance por entorno, así que los poderes de despliegue del agente son una lista blanca en lugar de un script de shell con credenciales.
5. Escalona la autonomía por entorno. En desarrollo, el agente despliega libremente. En producción, el agente prepara el release y la persona responsable de release lo autoriza, con un hook que aplica la puerta de producción. Staging queda en algún punto intermedio.
6. El rollback debería ser la ruta más ensayada del pipeline: un único comando que el agente pueda ejecutar y que se practique regularmente en staging. El play de cerrar el bucle (Etapa 6: Maintenance) invoca este rollback cuando se incumple una banda de control, así que tiene que estar probado de antemano.

Así se ve (paso de pipeline)

```
- name: Triage failed build
  if: failure()
  run: >
    claude -p "Read the build log at out/build.log. Identify the most
    likely cause, say whether the failure looks flaky or real, and write a
    three-line summary for the PR thread." >> triage.md
```

CONSIDERACIONES DE GOBERNANZA

El principio rector es que el agente puede actuar hasta la puerta de producción y no puede atravesarla. Los controles siguientes hacen cumplir este principio:

- La protección de rama convierte todo lo que escribe el agente en un PR, sin ruta directa a `main`.
- El hook de despliegue a producción bloquea el release hasta que una persona nombrada como responsable de release lo autoriza. Cada ejecución no interactiva actúa bajo la identidad propia del agente, así que el log del pipeline separa lo que hizo el agente de lo que hizo la persona ingeniera que lo disparó.
- Los niveles de permiso por entorno determinan cuánto puede hacer el agente de camino a la puerta.

CÓMO MEDIRLO

INDICADOR ADELANTADO

El porcentaje de fallos de pipeline triados sin escalar a una persona, tomado de los logs del pipeline de CI/CD.

INDICADOR REZAGADO

Las métricas DORA (DevOps Research and Assessment), que el sistema de CI y las herramientas de despliegue ya emiten.

06 Maintain

El bucle se cierra. Un disparador invoca a Claude sin ninguna persona en la ruta de invocación, y lo que encuentra vuelve a entrar en el pipeline como un nuevo intent.md.

Mantenimiento y el cierre del bucle

Hasta ahora hemos hablado de cómo incorporar a Claude en cada etapa del proceso de SDLC, con cada etapa requiriendo que una persona lance los pasos iniciales. Esta etapa, en cambio, traslada el foco hacia la ejecución autónoma de Claude para cerrar el bucle.

Por ejemplo, un agente de monitoreo que corre de forma continua podría, a raíz de un ticket de bug, crear un `intent.md`, y avanzar por las fases de requisitos, plan, construcción, pruebas y revisión. La Etapa 6: Maintenance corre sin supervisión directa (*headless*), con una puerta de confianza independiente entre etapas —una comprobación determinista o un agente revisor adversarial— que decide si el resultado de la etapa anterior continúa o se escala a una persona.

Tradicional

El mantenimiento es una fase reactiva. Todos los tickets o incidentes esperan a que una persona actúe y reinicie el proceso. Una alerta suena a las 3 de la madrugada y puede pasar desapercibida, un ticket puede quedarse en el backlog hasta que alguien lo recoja, y las acciones de un post-mortem pueden no llegar nunca al código si estalla otro incendio antes.

Nativo de IA

Un disparador —como el incumplimiento de una banda de control, un ticket, un mensaje de canal o una programación— invoca a Claude sin ninguna persona en la ruta. Claude diagnostica, actúa solo a través de rutas con puertas de control, y escribe lo que encuentra como `intent.md`, que entonces atraviesa las etapas descritas antes. Las personas triángulan y revisan ese trabajo, y ya no tienen que iniciarlo.

Cerrar el bucle

Un script determinista vigila producción e invoca a Claude cuando se incumple una banda de control. El monitoreo de un incumplimiento es un ejemplo útil del patrón para que el bucle corra de forma autónoma, mientras que la sección de Claude Tag (beta pública), al final de esta etapa, cubre el trabajo que llega por otros canales.

CÓMO EMPEZAR

REQUISITOS PREVIOS

`intent.md`, que le da al bucle una salida estructurada para reiniciar. Las revisiones de PR aceleradas por Claude, los hooks como límite de acción, y una ruta de rollback para CI/CD (que invoca el nivel más alto de autonomía).

INFRAESTRUCTURA

Un almacén de métricas que el script de detección pueda consultar (Prometheus, la API del sistema de CI, o equivalentes), acceso de lectura al repositorio, una forma de ejecutar Claude Code de forma no interactiva en CI, o el Agent SDK para un servicio que reciba webhooks.

CÓMO EJECUTARLO

1. Quien es dueño del servicio, o la persona de plataforma, elige una métrica con una línea base móvil estable, como la tasa de fallos de pruebas en CI, la tasa de errores 5xx tras el despliegue, o el tiempo de ciclo de los PR.
2. Escriben el script de detección, típicamente media y desviación estándar sobre una ventana móvil con reglas (Western Electric o similares) para que las bandas capturen tanto derivas lentas como picos. El script está bajo control de versiones y tiene pruebas unitarias, y la detección se mantiene completamente determinista, sin ningún modelo involucrado.
3. Los niveles de respuesta se definen en configuración versionada (`bands.yaml` abajo). En 1σ el script solo registra, en 2σ invoca a Claude en modo lectura para diagnosticar, y en 3σ Claude puede actuar, aunque solo abriendo un PR hacia la puerta de revisión o disparando un runbook preaprobado.
4. La capa de disparo puede ser un workflow programado en GitHub o GitLab, un webhook desde el stack de monitoreo existente, o un Cron Job dentro de la red. Claude se ejecuta sin estado, ya sea como un paso no interactivo en un runner de CI o como un servicio del Agent SDK en un contenedor en sandbox, y el play de CI/CD cubre las opciones de despliegue y acceso al modelo. Como la ejecución no tiene estado y es no interactiva, un bucle puede empezar y terminar sin que nadie lo inicie.
5. El agente escribe su diagnóstico como `intent.md` en el formato de la Etapa 1: Plan, cubriendo la anomalía y su evidencia, un resultado propuesto, los sistemas afectados y cualquier pregunta abierta. Desde ahí, el hallazgo atraviesa el pipeline como cualquier otro.
6. Quien es dueño del servicio, o la persona de guardia, triángula la cola, enrutando los hallazgos de cara al producto hacia el product owner: corregir ahora, programar, o descartar. Los descartes afinan las bandas y ayudan a reducir el ruido.

7. Cuando se publica una corrección, se añade un eval para ese incidente (el play de evaluaciones continuas), para asegurar que ese tipo de problema quede protegido en adelante.

Así se ve (por ejemplo, un bands.yaml que monitoriza la tasa de fallos de pruebas en CI)

```
metric: ci_test_failure_rate
baseline: rolling_30d
rules: western_electric
tiers:
  1sigma: { action: log }
  2sigma: { action: diagnose,
            tools: "Read,Grep,Bash(gh run view *)" }
  3sigma: { action: propose,
            routes: [pull_request, runbook:rollback-deploy] }
```

CONSIDERACIONES DE GOBERNANZA

Los límites de los niveles se aplican desde configuración versionada, con permisos y configuración gestionada que niegan el acceso a producción. Las invocaciones, hallazgos y decisiones de triaje quedan registrados con marca de tiempo. Quien es dueño del servicio triángula y aprueba los hallazgos, los cambios resultantes pasan por la puerta normal de revisión de PR, y los runbooks que el agente puede disparar fueron aprobados de antemano.

CÓMO MEDIRLO

INDICADOR ADELANTADO

Tiempo desde el incumplimiento de la banda hasta un `intent.md` en la cola de triaje, frente al viejo tiempo desde el incidente hasta la acción de post-mortem. El log del script de detección tiene la marca de tiempo del incumplimiento y el nivel del incidente.

INDICADOR REZAGADO

El porcentaje de hallazgos que se convierten en correcciones fusionadas (cola de triaje frente al historial real de PRs), y los incidentes repetidos de la misma clase, que deberían caer a medida que las correcciones se suman a la suite de evals.

EJEMPLOS

- Cuando la tasa de fallos de pruebas en CI incumple 3σ, el agente pone en cuarentena la prueba inestable o abre un PR de reversión, y la puerta de revisión decide.
- Cuando la tasa de errores 5xx tras el despliegue incumple 3σ con un despliegue dentro de la ventana, el agente dispara el pipeline de rollback existente.
- Cuando el tiempo de ciclo de los PR activa una regla de deriva, el agente escribe un informe para la dirección de ingeniería, lo que demuestra que el mismo arnés funciona tanto para

métricas de proceso como de producción.

La detección se mantiene determinista. Claude se invoca solo una vez incumplida una banda, y el nivel define qué puede hacer.

Escaneos recurrentes de la base de código

Un escaneo de seguridad es una declaración puntual sobre una base de código bajo un modelo concreto, y las dos mitades caducan: el código cambia cada semana, y cada generación de modelo encuentra vulnerabilidades que la anterior no vio. La respuesta nativa de IA es ejecutar el escaneo según un horario, sin ninguna persona en la ruta de invocación, y enviar lo que encuentra a través de las mismas puertas que cualquier otro cambio en la base de código.

Claude Security es la forma alojada de escaneo programado. Se conecta un repositorio de GitHub, y los escaneos corren sobre Claude Mythos 5 en la infraestructura de Anthropic, con cada hallazgo validado antes de reportarse y con una calificación de confianza adjunta. Los parches sugeridos se revisan y aplican en Claude Code en la web. La organización obtiene los hallazgos sin necesitar acceso al modelo en sí.

Tradicional

El escaneo de seguridad es un evento puntual: se lanza un escaneo antes de un release o una auditoría. El informe va a un rastreador, y el backlog se trabaja a mano hasta el siguiente evento. El código escrito entre medias queda cubierto por lo que haya captado la revisión de PR.

Nativo de IA

Los escaneos corren según un horario contra cada repositorio conectado, sobre el modelo más capaz disponible, con los hallazgos validados antes de que nadie los lea. Cada hallazgo se trata como una banda de control incumplida: una corrección que cabe en un PR pasa por la puerta de revisión, y cualquier cosa más grande se convierte en un `intent.md`. La cobertura tiene fecha de la última ejecución, no de la primera.

CÓMO EMPEZAR

REQUISITOS PREVIOS

La puerta de revisión de PR y los hooks como puertas de aprobación (Etapa 5: Deploy), para que los hallazgos pasen por revisión como cualquier otro cambio. El formato `intent.md` de la Etapa 1: Plan para hallazgos demasiado grandes para un solo PR.

INFRAESTRUCTURA

Claude Security está disponible para organizaciones de Claude Enterprise en beta pública. Necesita la app de GitHub de Anthropic instalada en los repositorios objetivo (alojados en la nube en github.com), Claude Code on the Web habilitado, Extra Usage activado con un límite de gasto definido, asientos premium para quienes ejecutan escaneos, y la función activada por una persona admin en `claude.ai/admin-settings/claude-code`. Los escaneos se facturan por consumo a las tarifas de Mythos 5, así que el límite de gasto debería ajustarse al tamaño y número de repositorios.

CÓMO EJECUTARLO

1. La persona líder de seguridad conecta los repositorios y los organiza en proyectos por repositorio, servicio o equipo, para que la propiedad de los hallazgos quede clara desde el principio.
2. Se ejecuta un primer escaneo completo de los repositorios más críticos, incluidos los que ya fueron escaneados antes por otras herramientas o por modelos anteriores. Este primer escaneo se trata como línea base, y es probable que encuentre hallazgos en código que se consideraba limpio.
3. Se fija un horario por proyecto. Semanal es un valor por defecto razonable para servicios en desarrollo activo; los escaneos se acotan a un directorio o rama cuando un repositorio es grande o mixto.
4. Se triángulan los hallazgos con la calificación de confianza en mano. Se descarta con una razón, así que el descarte queda registrado y el mismo hallazgo no vuelve a aparecer como nuevo en la siguiente ejecución.
5. Para un hallazgo acotado, se abre el parche sugerido en Claude Code on the Web, se revisa, y se envía a través de la puerta de revisión de PR como cualquier otro cambio. El agente que propuso la corrección no tiene ninguna ruta para aprobarla.
6. Para algo más amplio que un solo parche, como una debilidad arquitectónica o un patrón repetido entre servicios, se redacta como `intent.md` en el formato de la Etapa 1 y se arranca en Plan.

7. Cuando una corrección se libera a producción, se añade un eval para esa clase de vulnerabilidad a la suite del play de evaluaciones continuas, para que la configuración que guía al agente quede probada contra esa clase desde entonces.
8. Se exportan los hallazgos como CSV o Markdown, o se usan webhooks, para mantener el rastreador y los sistemas de auditoría existentes de la organización como el sistema de registro donde ya los esperan las auditorías.

CONSIDERACIONES DE GOBERNANZA

El escaneo corre bajo los controles de administración de la organización, es decir: qué repositorios están conectados, quién tiene un asiento de escaneo, y el límite de gasto se fijan de forma centralizada. Cada hallazgo tiene un resultado de validación y una calificación de confianza, y cada descarte tiene una razón, así que el historial de escaneos es un registro de auditoría de lo que se encontró, se corrigió, y se aceptó conscientemente.

Las correcciones llegan a producción a través de la puerta de revisión de PR y la protección de rama, no directamente desde el escaneo. Claude Security complementa el análisis estático y el escaneo de dependencias existentes: las comprobaciones deterministas se quedan en CI, y el escaneo guiado por modelo cubre las vulnerabilidades dependientes de contexto que esas comprobaciones no están diseñadas para encontrar.

CÓMO MEDIRLO

INDICADOR ADELANTADO

Porcentaje de repositorios conectados con un horario activo, y el tiempo desde que se reporta un hallazgo hasta que su parche entra en la puerta de revisión de PR, leído del historial de escaneos y los metadatos del PR.

INDICADOR REZAGADO

Vulnerabilidades encontradas por el escaneo programado frente a las encontradas en producción o por reporte externo, desde el rastreador de incidentes; y la tendencia de hallazgos por escaneo en repositorios que ya han pasado por varias ejecuciones, que debería caer a medida que se acumulan correcciones y evals.

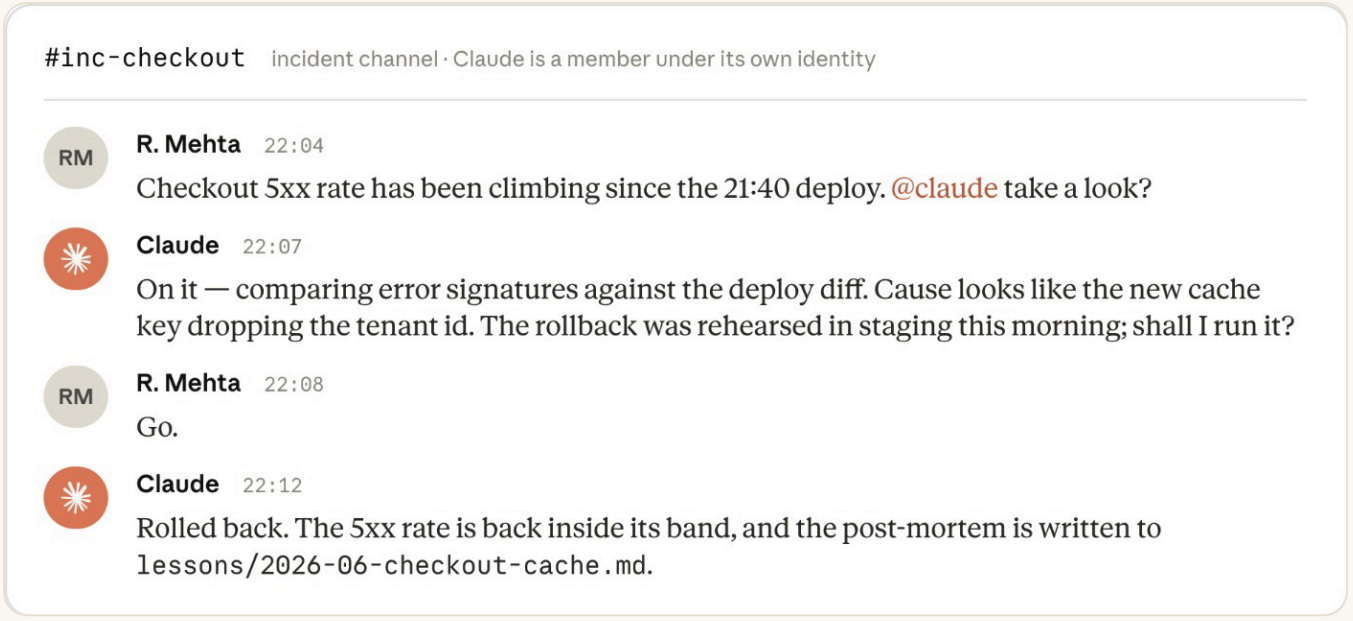
Claude de guardia con Claude Tag

Los incidentes también pueden llegar por otros medios, como apps de comunicación laboral, tipo Slack o Teams. Un incidente puede verse como un mensaje de Slack a las 10 de la noche pidiendo una corrección urgente en un canal de incidentes, y ahora puede accionarse de inmediato. Claude Tag (beta

pública, disponible actualmente en Slack) convierte a Claude en miembro de esos canales bajo su propia identidad, así que cada incidente nuevo obtiene un primer respondiente, y la propia respuesta pasa a formar parte del bucle y de la memoria para incidentes futuros.

La conversación y el conocimiento institucional permanecen en el canal, con cualquier persona del canal capaz de guiar y accionar la respuesta. Cualquier miembro del equipo puede probar hipótesis, explorar nuevas opciones e investigar en tiempo real, con el historial del canal sumando a la auditabilidad. A través del acceso a MCP, Claude verifica que la métrica volvió a su línea base y lo confirma en el hilo, y escribe el post-mortem en un archivo de lecciones aprendidas versionado que investigaciones futuras pueden leer.

Los incidentes no son el único trabajo que recoge Claude Tag. Etiquetado en un ticket vía MCP, o consultado en el canal, Claude triángula el trabajo de la misma manera. Una corrección pequeña y bien acotada llega como un PR a través de la puerta de revisión, y cualquier cosa más grande se redacta como `intent.md` para la Etapa 1: Plan, momento en el que el bucle empieza a retroalimentarse solo. Ver: *how Claude Tag runs on-call for CI/CD at Anthropic*.



El canal es el rastro de auditoría: la petición, el diagnóstico, la autorización humana y la corrección permanecen todos donde se gestionó el incidente.

Reflexiones finales

Los modelos y los harnesses se han vuelto más avanzados, lo que permite a las organizaciones transformar no solo cómo producen código, sino todo el ciclo de vida de desarrollo de software.

Esta transformación mantiene el criterio humano en el centro del proceso y tiene en cuenta los requisitos de gobernanza y regulación de las grandes organizaciones empresariales.

Esta guía reúne muchas de las mejores prácticas reales que nuestro equipo de Applied AI ejecuta a diario con nuestros clientes, y esperamos que te resulte un recurso práctico y accionable.

El bucle sigue corriendo. El criterio humano permanece por encima de él.

Recursos y agradecimientos

La siguiente documentación es lo que necesita un equipo de plataforma para configurar estos controles, más o menos en el orden en que los desplegarías.

- Configurar Claude Code para tu organización — el mapa de decisiones para administradores; el punto de partida
  `code.claude.com/docs/en/admin-setup`
- Referencia y precedencia de settings, incluida cada clave exclusiva de configuración gestionada
  `code.claude.com/docs/en/settings`
- Settings gestionados desde la consola de administración de Claude
  `code.claude.com/docs/en/server-managed-settings`
- Permisos
  `code.claude.com/docs/en/permissions`
- Sandboxing — aislamiento de sistema de archivos y red a nivel de SO
  `code.claude.com/docs/en/sandboxing`
- Hooks — guía
  `code.claude.com/docs/en/hooks-guide`

- Hooks — referencia
  `code.claude.com/docs/en/hooks`
- Skills
  `code.claude.com/docs/en/skills`
- Plugins y marketplaces privados — cómo se distribuyen skills y hooks en toda la organización
  `code.claude.com/docs/en/plugin-marketplaces`
- MCP gestionado — control centralizado de la superficie de herramientas del agente
  `code.claude.com/docs/en/managed-mcp`
- Resumen de despliegue empresarial — Bedrock, Vertex, Foundry
  `code.claude.com/docs/en/third-party-integrations`
- Configuración de red empresarial
  `code.claude.com/docs/en/network-config`
- Monitoreo (OpenTelemetry)
  `code.claude.com/docs/en/monitoring-usage`
- El panel de analíticas
  `code.claude.com/docs/en/analytics`
- API de cumplimiento — feed de actividad empresarial, recuperación y borrado de chats
  `platform.claude.com/docs/en/manage-claude/compliance-api`
- Modelo de seguridad
  `code.claude.com/docs/en/security`

Gracias a Jim Blackhurst, Will Steuk y Jamal Arif por sus contribuciones a esta guía, inspirada y construida en buena parte sobre su trabajo previo.

Artículo original: claude.com/blog/the-ai-native-sdlc-playbook — Louis Claxton, 21 de agosto de 2026. Traducción y diagramación al español realizadas a partir del artículo original de Anthropic.